GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Proyecto: Sistema de Gestión de Pedidos Bocatto

Grupo 5 · NRC 36838 · Docente: Ing. Jenny Ruiz R. · 22 de octubre de 2026

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base	Gestión de pedidos del restaurante Bocatto Valley
Casos de uso	Tomar pedido en mesa, Realizar pedido por QR, Gestionar preparación del pedido y Cobrar pedido en la caja
Referencia base	01_Guia_PlantUML_CU_Bocatto_G5 y ERS_Bocatto_G5
Propósito	Guiar al estudiante para pasar del análisis funcional al análisis estructural y dinámico, manteniendo coherencia entre requisito, caso de uso, clase y secuencia.

1. Propósito de la guía

Esta guía complementa la guía inicial de casos de uso. Su finalidad es mostrar cómo el sistema de gestión de pedidos Bocatto puede analizarse de forma progresiva mediante tres modelos conectados: casos de uso de análisis, clases de análisis y diagramas de secuencia de análisis.

El énfasis no está en programar la interfaz, sino en comprender qué responsabilidad cumple cada elemento del modelo y cómo se conserva la trazabilidad desde el requisito funcional hasta la interacción entre objetos.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el actor con el sistema?	Diagrama de casos de uso y matriz RF-CU.	Permite identificar responsabilidades, clases frontera, control y entidad.
Clases de análisis	¿Qué objetos participan para cumplir los casos de uso?	Diagrama de clases de análisis con estereotipos boundary, control, entity y repositorio.	Define los objetos que aparecerán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo colaboran actor, frontera, control y entidad para ejecutar cada caso de uso?	Diagramas de secuencia por cada caso de uso de nivel 0 seleccionado.	Valida que el flujo sea coherente con los casos de uso y las clases identificadas.

3. Caso base actualizado: Gestión de pedidos Bocatto

El caso se basa en los cuatro casos de uso de nivel 0 trabajados en la guía 1. Para cada caso de uso se presenta una tabla con los datos que maneja, según la ERS, y otra con las acciones de interfaz que ejerce el actor, que corresponden a sus requisitos funcionales.

3.1 Caso de uso: Tomar pedido en mesa

Dato	Tipo sugerido	Descripción	Validación mínima

Número de mesa	Entero	Mesa numerada que el mesero selecciona para el pedido.	Obligatorio; la mesa debe existir.
Productos y cantidades	Lista	Productos solicitados por el cliente con su cantidad.	Al menos un producto.
Observaciones	Texto	Indicaciones del cliente, por ejemplo “sin cebolla” o “sin azúcar”.	Opcional.
Identificador del pedido	Entero o texto corto	Identificador interno que el sistema asigna al pedido.	Único y generado automáticamente.
Estado	Lista de valores	Estado inicial del pedido registrado.	Inicia en “pendiente”.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Seleccionar mesa	Tomar pedido en mesa	Asocia el pedido a una mesa numerada del restaurante.
Agregar productos y cantidades	Tomar pedido en mesa	Registra los productos solicitados por el cliente con sus cantidades.
Registrar observaciones	Tomar pedido en mesa	Guarda las indicaciones del cliente para la cocina.
Registrar pedido	Tomar pedido en mesa	Valida que haya al menos un producto, genera el identificador, deja el pedido en “pendiente” y lo envía a cocina.

3.2 Caso de uso: Realizar pedido por QR

Dato	Tipo sugerido	Descripción	Validación mínima
Código QR de la mesa	Texto	Código ubicado en la mesa que el cliente escanea.	Debe estar asignado a una mesa.
Número de mesa	Entero	Mesa que el sistema identifica a partir del QR.	Se obtiene automáticamente del QR.
Productos y cantidades	Lista	Productos del menú digital, con precios y promociones vigentes, que el cliente selecciona.	Al menos un producto.
Observaciones	Texto	Indicaciones que el cliente agrega a su pedido.	Opcional.
Estado	Lista de valores	Estado inicial del pedido registrado.	Inicia en “pendiente”.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Escanear QR	Realizar pedido por QR	El sistema identifica la mesa asociada al código QR.
Ver menú digital	Realizar pedido por QR	Muestra los productos disponibles, sus precios y las promociones vigentes.
Seleccionar productos y cantidades	Realizar pedido por QR	El cliente arma su pedido y puede agregar observaciones.
Confirmar pedido	Realizar pedido por QR	Registra el pedido asociado a la mesa con estado “pendiente” y lo envía a cocina.

3.3 Caso de uso: Gestionar preparación del pedido

Dato	Tipo sugerido	Descripción	Validación mínima
Identificador del pedido	Entero o texto corto	Pedido pendiente que el cocinero selecciona.	Debe existir y estar registrado.

Estado actual	Lista de valores	Estado en el que se encuentra el pedido.	Pendiente, en preparación o listo.
Nuevo estado	Lista de valores	Estado al que pasa el pedido conforme avanza la preparación.	En preparación, listo o listo para entregar, en secuencia válida.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Ver pedidos pendientes	Gestionar preparación del pedido	Muestra la lista de pedidos que esperan preparación.
Seleccionar pedido	Gestionar preparación del pedido	El cocinero elige el pedido que va a preparar.
Cambiar estado	Gestionar preparación del pedido	Actualiza el estado en secuencia válida; el cambio se guarda y lo ven el personal y el cliente.

3.4 Caso de uso: Cobrar pedido en la caja

Dato	Tipo sugerido	Descripción	Validación mínima
Número de mesa	Entero	Mesa que el cliente indica en caja para buscar su pedido.	Obligatorio; la mesa debe existir.
Detalle del pedido	Lista	Productos y cantidades del pedido asociado a la mesa.	El pedido debe estar en estado listo.
Total a pagar	Decimal	Valor total del pedido.	Calculado por el sistema.
Método de pago	Lista de valores	Efectivo o digital/transferencia.	Obligatorio para registrar el pago.
Estado	Lista de valores	Estado del pedido luego del cobro.	Queda como “pagado” y habilitado como “listo para entregar”.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Buscar pedido por mesa	Cobrar pedido en la caja	Recupera el pedido asociado al número de mesa indicado por el cliente.
Verificar detalle y total	Cobrar pedido en la caja	Muestra los productos del pedido y el total a pagar.
Seleccionar método de pago	Cobrar pedido en la caja	Registra si el cliente paga en efectivo o por medio digital/transferencia.
Registrar pago	Cobrar pedido en la caja	Marca el pedido como “pagado” y lo habilita como “listo para entregar”.

4. Requisitos funcionales de nivel 0

4.1 Caso de uso: Tomar pedido en mesa

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-02	El sistema debe permitir al mesero registrar pedidos presenciales asociados a una mesa numerada, con productos, cantidades y observaciones opcionales, validando que tenga al menos un producto y asignando un identificador único con estado “pendiente”.	Mesero	Tomar pedido en mesa

4.2 Caso de uso: Realizar pedido por QR

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-04	El sistema debe permitir que el cliente registre pedidos desde su celular escaneando el QR de la mesa, mostrando el menú digital y registrando el pedido con estado “pendiente”.	Cliente	Realizar pedido por QR

4.3 Caso de uso: Gestionar preparación del pedido

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-05	El sistema debe permitir al cocinero visualizar los pedidos pendientes y actualizar su estado en secuencia válida: pendiente, en preparación, listo y listo para entregar.	Cocinero	Gestionar preparación del pedido

4.4 Caso de uso: Cobrar pedido en la caja

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-06	El sistema debe permitir al cajero buscar pedidos por número de mesa, ver su detalle y registrar el método de pago, validando que el pedido esté listo antes del cobro.	Cajero	Cobrar pedido en la caja

5. Diagrama de casos de uso de análisis

El diagrama de casos de uso representa las funciones visibles para cada actor. El Mesero toma el pedido en la mesa, el Cliente lo realiza por QR, el Cocinero gestiona su preparación y el Cajero realiza el cobro. No se usan relaciones <<include>> ni <<extend>> porque la ERS no las define; la validación de al menos un producto es un paso dentro de cada caso de uso.

Código PlantUML sugerido

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Mesero" as Mesero
actor "Cliente" as Cliente
actor "Cocinero" as Cocinero
actor "Cajero" as Cajero

rectangle "Sistema de Gestión de Pedidos Bocatto" {
  usecase "Tomar pedido en mesa" as UC1
  usecase "Realizar pedido por QR" as UC2
  usecase "Gestionar preparación del pedido" as UC3
  usecase "Cobrar pedido en la caja" as UC4
}

Mesero --> UC1
Cliente --> UC2
Cocinero --> UC3
Cajero --> UC4

note right of UC2
Datos mínimos del pedido:
- Número de mesa
- Productos y cantidades (al menos uno)
- Observaciones (opcional)
- Estado inicial: pendiente
end note
@enduml
```

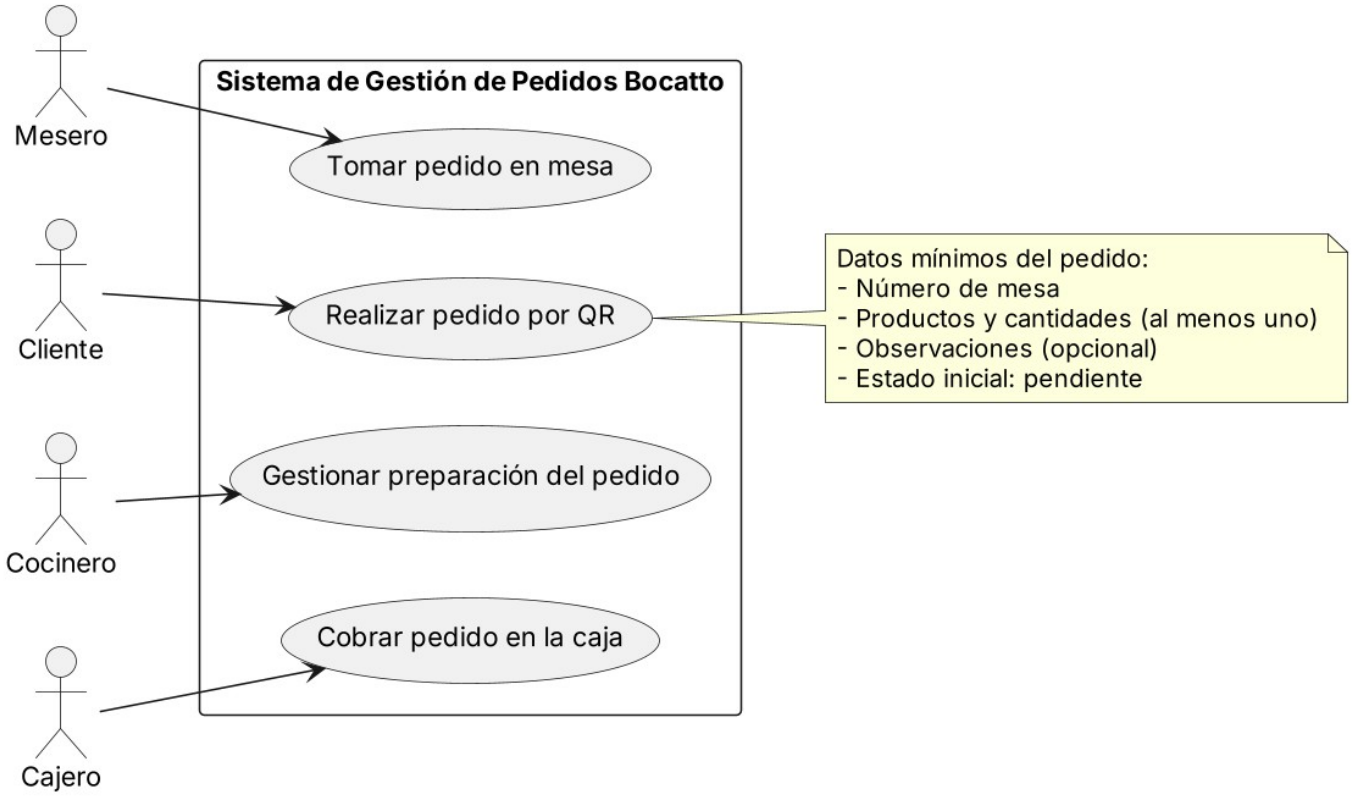


Figura 1. Diagrama de casos de uso nivel 0 del sistema Bocatto

6. Derivación de clases de análisis

A partir de los cuatro casos de uso se identifican clases de análisis con los estereotipos boundary, control y entity. Cada caso de uso tiene su propia frontera y su propio control, mientras que las entidades Pedido, Mesa y Producto se comparten. Además, se incluye RepositorioPedido para representar dónde se guardan y consultan los pedidos, sin entrar aún en detalles técnicos de base de datos.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con los casos de uso
Boundary / Frontera	Recibe datos del actor y muestra resultados.	PantallaPedido, PantallaMenuDigital, PantallaCocina, PantallaCaja	Representan la pantalla del mesero, el menú digital del cliente, la pantalla de cocina y la pantalla de caja.
Control	Coordina la lógica de cada caso de uso.	ControlPedido, ControlPedidoQR, ControlPreparacion, ControlCobro	Validan el pedido, registran pedidos, controlan la secuencia de estados y registran el pago.
Entity / Entidad	Representa la información del dominio.	Pedido, Mesa, Producto	Pedido contiene mesa, productos, cantidades, observaciones, estado y método de pago; Mesa y Producto apoyan el registro.
Repositorio / Colección	Conserva y recupera registros.	RepositorioPedido	Permite guardar, buscar por ID o por mesa, listar pendientes y actualizar pedidos.

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false

class "PantallaPedido" <<boundary>> {
  - numeroMesa: int
  - productosSeleccionados: List<Producto>
  - observaciones: String
  + seleccionarMesa(numeroMesa: int)
  + agregarProducto(producto: Producto, cantidad: int)
  + clickRegistrarPedido()
  + mostrarMensaje(mensaje: String)
}

class "PantallaMenuDigital" <<boundary>> {
  - codigoQR: String
  + escanearQR(codigoQR: String)
  + mostrarMenu(productos: List<Producto>)
  + seleccionarProducto(producto: Producto, cantidad: int)
  + clickConfirmarPedido()
  + mostrarMensaje(mensaje: String)
}

class "PantallaCocina" <<boundary>> {
  + mostrarPendientes(pedidos: List<Pedido>)
  + seleccionarPedido(idPedido: String)
  + clickCambiarEstado(nuevoEstado: String)
  + notificarNuevoPedido(pedido: Pedido)
  + mostrarMensaje(mensaje: String)
}

class "PantallaCaja" <<boundary>> {
  - numeroMesa: int
  + buscarPorMesa(numeroMesa: int)
  + mostrarDetalle(pedido: Pedido)
  + seleccionarMetodoPago(metodoPago: String)
  + clickRegistrarPago()
  + mostrarMensaje(mensaje: String)
}

class "ControlPedido" <<control>> {
  + registrarPedido(numeroMesa: int, productos: List<Producto>, cantidades: List<int>, observaciones:
String): Resultado
  + validarPedido(productos: List<Producto>): boolean
}

class "ControlPedidoQR" <<control>> {
  + obtenerMenu(codigoQR: String): List<Producto>
  + registrarPedidoQR(codigoQR: String, productos: List<Producto>, cantidades: List<int>, observaciones:
String): Resultado
  + validarPedido(productos: List<Producto>): boolean
}

class "ControlPreparacion" <<control>> {
  + listarPendientes(): List<Pedido>
  + cambiarEstado(idPedido: String, nuevoEstado: String): Resultado
  + validarSecuencia(estadoActual: String, nuevoEstado: String): boolean
}

class "ControlCobro" <<control>> {
  + buscarPedidoPorMesa(numeroMesa: int): Pedido
  + registrarPago(idPedido: String, metodoPago: String): Resultado
  + validarPedidoListo(pedido: Pedido): boolean
}

class "Pedido" <<entity>> {
  - id: String
  - numeroMesa: int
  - productos: List<Producto>
  - cantidades: List<int>
  - observaciones: String
  - estado: String
  - metodoPago: String
```

```
  - total: double
  + Pedido(numeroMesa: int, productos: List<Producto>, cantidades: List<int>, observaciones: String)
  + cambiarEstado(nuevoEstado: String): void
  + calcularTotal(): double
  + registrarPago(metodoPago: String): void
  + getEstado(): String
}

class "Mesa" <<entity>> {
  - numero: int
  - codigoQR: String
  + getNumero(): int
  + getCodigoQR(): String
  + validarNumero(numero: int): boolean
}

class "Producto" <<entity>> {
  - id: String
  - nombre: String
  - precio: double
  - disponible: boolean
  + estaDisponible(): boolean
  + getPrecio(): double
}

class "RepositorioPedido" <<repository>> {
  + guardar(pedido: Pedido): void
  + buscarPorId(idPedido: String): Pedido
  + buscarPorMesa(numeroMesa: int): Pedido
  + listarPendientes(): List<Pedido>
  + actualizar(pedido: Pedido): void
}

PantallaPedido --> ControlPedido : solicita registro
PantallaMenuDigital --> ControlPedidoQR : solicita menú y registro
PantallaCocina --> ControlPreparacion : solicita cambio de estado
PantallaCaja --> ControlCobro : solicita cobro

ControlPedido --> Pedido : crea
ControlPedido --> RepositorioPedido : guarda
ControlPedidoQR --> Pedido : crea
ControlPedidoQR --> RepositorioPedido : guarda
ControlPreparacion --> Pedido : actualiza estado
ControlPreparacion --> RepositorioPedido : consulta y actualiza
ControlCobro --> Pedido : registra pago
ControlCobro --> RepositorioPedido : consulta y actualiza

ControlPedido ..> Mesa : verifica
ControlPedido ..> Producto : consulta
ControlPedido ..> PantallaCocina : notifica
ControlPedidoQR ..> Mesa : identifica
ControlPedidoQR ..> Producto : consulta
ControlPedidoQR ..> PantallaCocina : notifica
ControlCobro ..> Mesa : verifica

RepositorioPedido o-- "0..*" Pedido : almacena
Pedido --> "1" Mesa : se asocia a
Pedido --> "1..*" Producto : contiene
@enduml
```

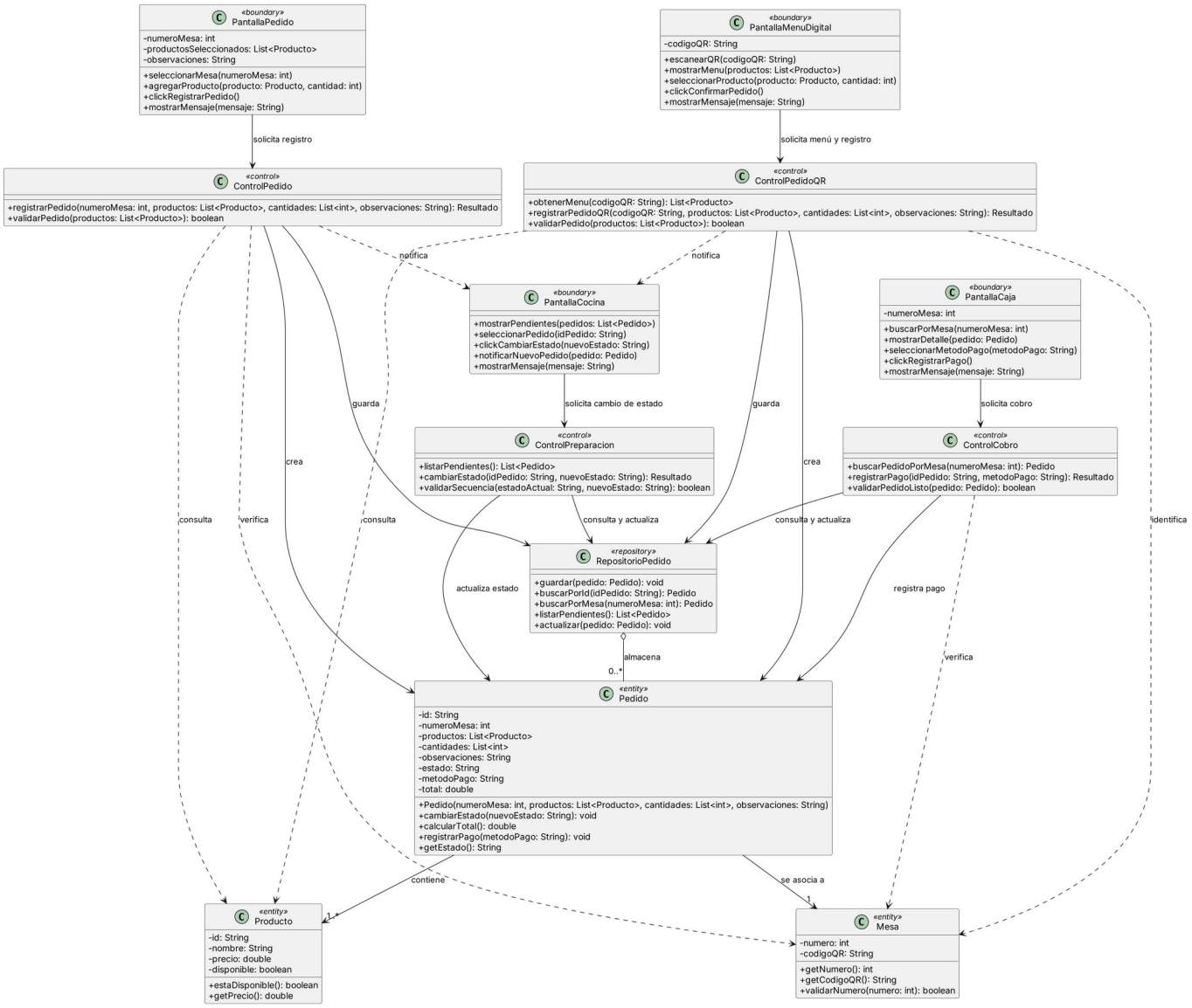


Figura 2. Diagrama de clases de análisis del sistema Bocatto (frontera, control, entidad y repositorio)

7. Matriz de relación: casos de uso, clases y secuencias

7.1 Caso de uso: Tomar pedido en mesa

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Tomar pedido en mesa	PantallaPedido captura mesa, productos, cantidades y observaciones.	ControlPedido verifica la mesa, consulta productos, valida el pedido y lo envía a cocina.	Crea Pedido (pendiente) y lo guarda en RepositorioPedido; usa Mesa y Producto.	Secuencia: seleccionar mesa -> agregar productos -> validar -> crear pedido -> guardar -> notificar a cocina -> confirmar.

7.2 Caso de uso: Realizar pedido por QR

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Realizar pedido por QR	PantallaMenuDigital lee el QR, muestra el menú y captura la selección.	ControlPedidoQR identifica la mesa, obtiene el menú, valida y registra el pedido.	Crea Pedido (pendiente) y lo guarda en RepositorioPedido; usa Mesa y Producto.	Secuencia: escanear QR -> mostrar menú -> seleccionar -> confirmar -> validar -> guardar -> notificar a cocina.

7.3 Caso de uso: Gestionar preparación del pedido

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Gestionar preparación del pedido	PantallaCocina muestra pendientes y recibe el nuevo estado.	ControlPreparacion lista pendientes y valida la secuencia de estados.	Actualiza el estado de Pedido en RepositorioPedido.	Secuencia: listar pendientes -> seleccionar -> validar secuencia -> cambiar estado -> actualizar -> confirmar.

7.4 Caso de uso: Cobrar pedido en la caja

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Cobrar pedido en la caja	PantallaCaja busca por mesa, muestra detalle y total y recibe el método de pago.	ControlCobro verifica la mesa, valida que el pedido esté listo y registra el pago.	Consulta Pedido en RepositorioPedido, registra el pago y lo actualiza.	Secuencia: buscar por mesa -> validar listo -> mostrar total -> registrar pago -> actualizar -> confirmar.

8. Diagramas de secuencia de análisis

Los diagramas de secuencia muestran cómo colaboran los objetos de análisis en el tiempo. Se elaboró un diagrama por cada uno de los cuatro casos de uso de nivel 0 para evitar diagramas sobrecargados.

8.1 Secuencia: Tomar pedido en mesa

```
@startuml
title Secuencia de análisis - Tomar pedido en mesa
actor "Mesero" as Mesero
boundary "PantallaPedido" as UI
control "ControlPedido" as Ctrl
entity "Mesa" as Mesa
entity "Producto" as Prod
entity "Pedido" as Ped
database "RepositorioPedido" as Repo
boundary "PantallaCocina" as UICocina

Mesero -> UI : seleccionarMesa(numeroMesa)
Mesero -> UI : agregarProducto(producto, cantidad)
Mesero -> UI : clickRegistrarPedido()
UI -> Ctrl : registrarPedido(numeroMesa, productos, cantidades, observaciones)
Ctrl -> Mesa : validarNumero(numeroMesa)
Mesa --> Ctrl : true
Ctrl -> Prod : estaDisponible()
Prod --> Ctrl : true
Ctrl -> Ctrl : validarPedido(productos)
Ctrl -> Ped : new Pedido(numeroMesa, productos, cantidades, observaciones)
note right of Ped : estado = "pendiente"
Ctrl -> Repo : guardar(Ped)
Repo --> Ctrl : guardado correcto
Ctrl -> UICocina : notificarNuevoPedido(Ped)
Ctrl --> UI : Resultado exitoso
```

```
UI --> Mesero : mostrarMensaje("Pedido registrado")
@enduml
```

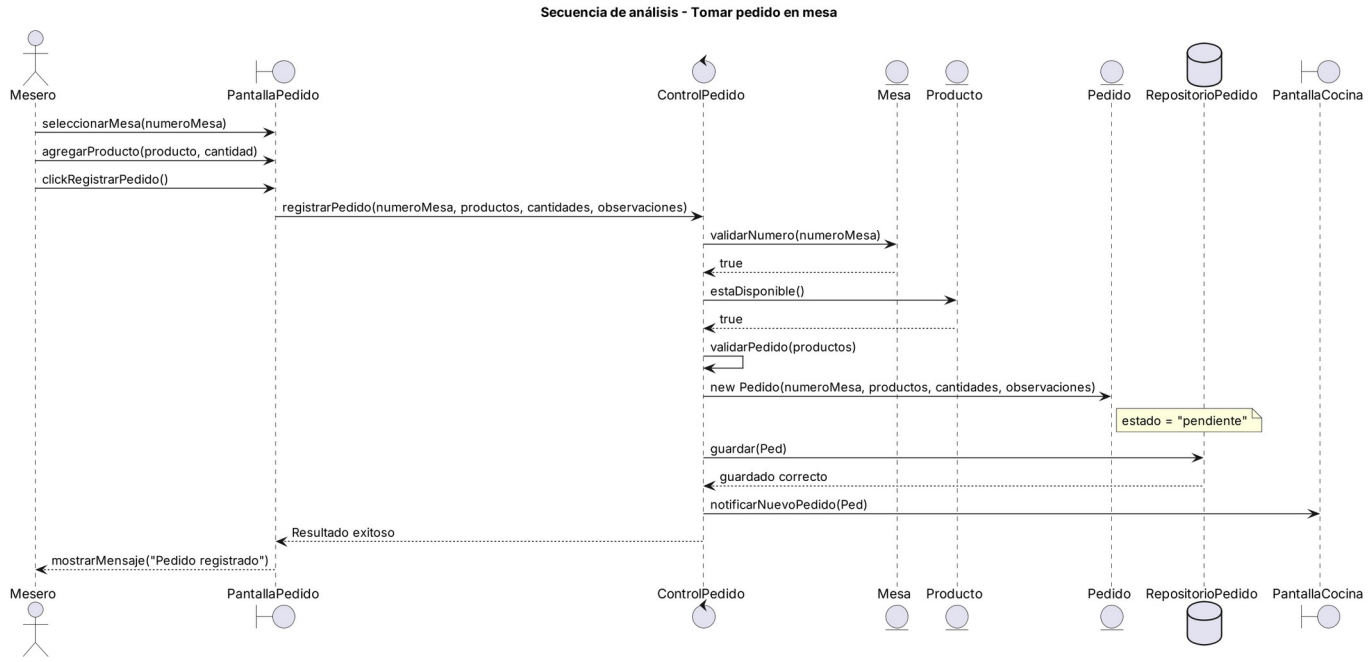


Figura 3. Secuencia de análisis: Tomar pedido en mesa

8.2 Secuencia: Realizar pedido por QR

```
@startuml
title Secuencia de análisis - Realizar pedido por QR
actor "Cliente" as Cliente
boundary "PantallaMenuDigital" as UI
control "ControlPedidoQR" as Ctrl
entity "Mesa" as Mesa
entity "Producto" as Prod
entity "Pedido" as Ped
database "RepositorioPedido" as Repo
boundary "PantallaCocina" as UICocina

Cliente -> UI : escanearQR(codigoQR)
UI -> Ctrl : obtenerMenu(codigoQR)
Ctrl -> Mesa : getNumero()
Mesa --> Ctrl : numeroMesa
Ctrl -> Prod : estaDisponible()
Prod --> Ctrl : productos disponibles
Ctrl --> UI : listaProductos
UI --> Cliente : mostrarMenu(productos)
Cliente -> UI : seleccionarProducto(producto, cantidad)
Cliente -> UI : clickConfirmarPedido()
UI -> Ctrl : registrarPedidoQR(codigoQR, productos, cantidades, observaciones)
Ctrl -> Ctrl : validarPedido(productos)
Ctrl -> Ped : new Pedido(numeroMesa, productos, cantidades, observaciones)
note right of Ped : estado = "pendiente"
Ctrl -> Repo : guardar(Ped)
Repo --> Ctrl : guardado correcto
Ctrl -> UICocina : notificarNuevoPedido(Ped)
Ctrl --> UI : Resultado exitoso
UI --> Cliente : mostrarMensaje("Pedido enviado a cocina")
@enduml
```

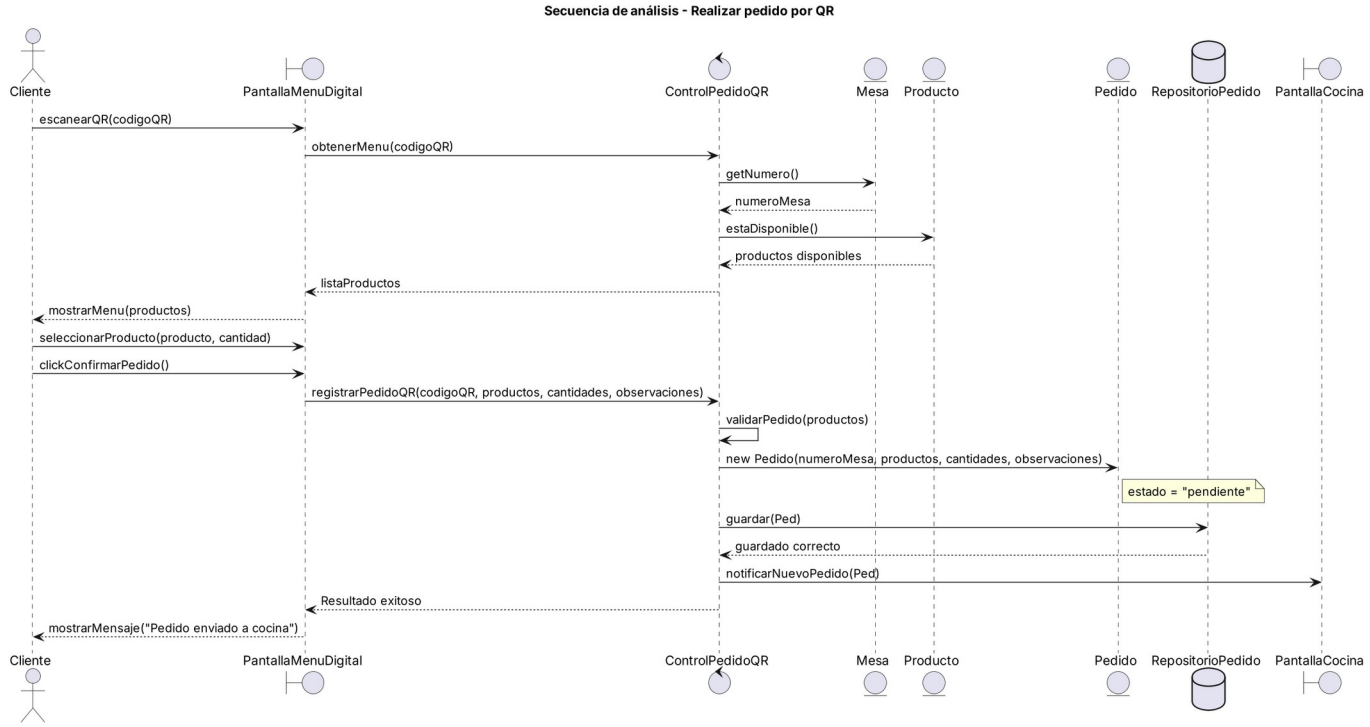


Figura 4. Secuencia de análisis: Realizar pedido por QR

8.3 Secuencia: Gestionar preparación del pedido

```
@startuml
title Secuencia de análisis - Gestionar preparación del pedido
actor "Cocinero" as Cocinero
boundary "PantallaCocina" as UI
control "ControlPreparacion" as Ctrl
database "RepositorioPedido" as Repo
entity "Pedido" as Ped

Cocinero -> UI : revisa pedidos pendientes
UI -> Ctrl : listarPendientes()
Ctrl -> Repo : listarPendientes()
Repo --> Ctrl : listaPedidos
Ctrl --> UI : listaPedidos
UI --> Cocinero : mostrarPendientes(pedidos)
Cocinero -> UI : seleccionarPedido(idPedido)
Cocinero -> UI : clickCambiarEstado(nuevoEstado)
UI -> Ctrl : cambiarEstado(idPedido, nuevoEstado)
Ctrl -> Repo : buscarPorId(idPedido)
Repo --> Ctrl : Pedido encontrado
Ctrl -> Ctrl : validarSecuencia(estadoActual, nuevoEstado)
Ctrl -> Ped : cambiarEstado(nuevoEstado)
note right of Ped : en preparación, listo\no listo para entregar
Ctrl -> Repo : actualizar(Ped)
Repo --> Ctrl : actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Cocinero : mostrarMensaje("Estado actualizado")
@enduml
```

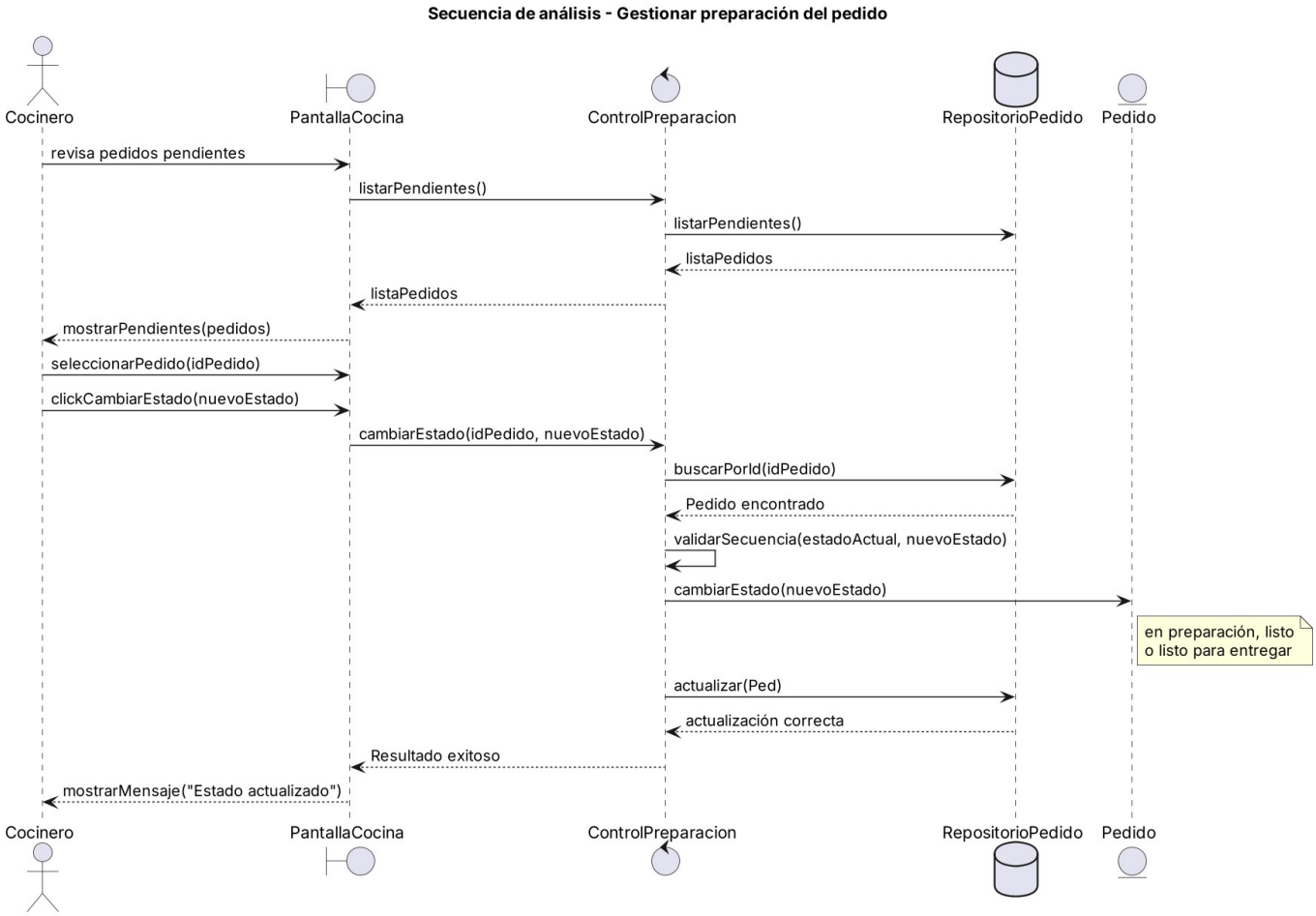


Figura 5. Secuencia de análisis: Gestionar preparación del pedido

8.4 Secuencia: Cobrar pedido en la caja

```
@startuml
title Secuencia de análisis - Cobrar pedido en la caja
actor "Cajero" as Cajero
boundary "PantallaCaja" as UI
control "ControlCobro" as Ctrl
entity "Mesa" as Mesa
database "RepositorioPedido" as Repo
entity "Pedido" as Ped

Cajero -> UI : buscarPorMesa(numeroMesa)
UI -> Ctrl : buscarPedidoPorMesa(numeroMesa)
Ctrl -> Mesa : validarNumero(numeroMesa)
Mesa --> Ctrl : true
Ctrl -> Repo : buscarPorMesa(numeroMesa)
Repo --> Ctrl : Pedido encontrado
Ctrl -> Ctrl : validarPedidoListo(Ped)
Ctrl -> Ped : calcularTotal()
Ped --> Ctrl : total
Ctrl --> UI : detalle y total
UI --> Cajero : mostrarDetalle(pedido)
Cajero -> UI : seleccionarMetodoPago(metodoPago)
Cajero -> UI : clickRegistrarPago()
UI -> Ctrl : registrarPago(idPedido, metodoPago)
Ctrl -> Ped : registrarPago(metodoPago)
note right of Ped : estado = "pagado"\nhabilitado para entregar
Ctrl -> Repo : actualizar(Ped)
Repo --> Ctrl : actualización correcta
```

```
Ctrl --> UI : Resultado exitoso
UI --> Cajero : mostrarMensaje("Pago registrado")
@enduml
```

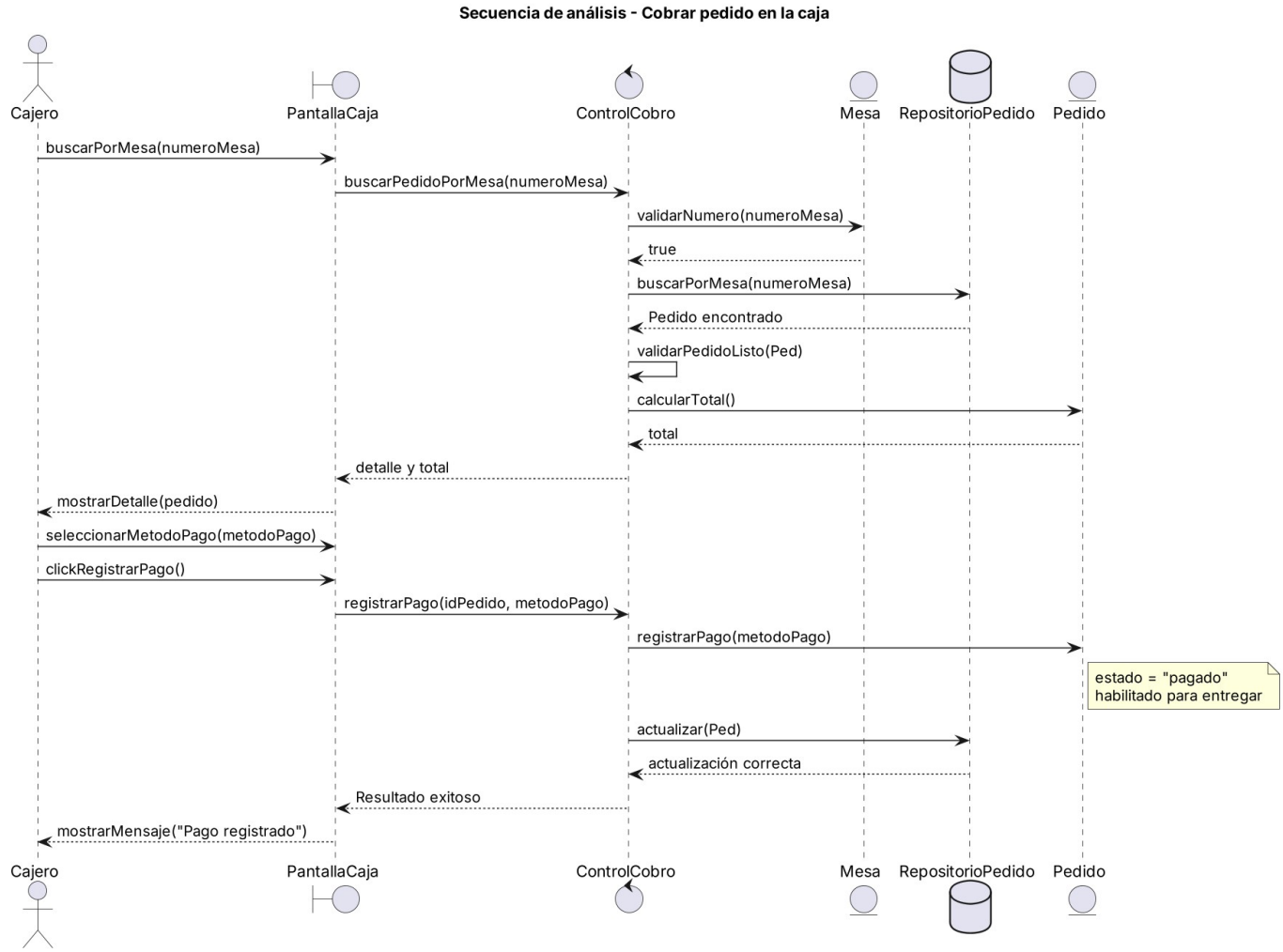


Figura 6. Secuencia de análisis: Cobrar pedido en la caja

9. Especificación breve de casos de uso conectada con las secuencias

9.1 Caso de uso: Tomar pedido en mesa

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Tomar pedido en mesa	Mesero con acceso al sistema y mesa numerada existente.	Seleccionar mesa -> registrar productos y cantidades -> agregar observaciones -> validar que tenga al menos un producto -> guardar pedido -> enviar a cocina.	Pedido con identificador y estado “pendiente” enviado a cocina.

9.2 Caso de uso: Realizar pedido por QR

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Realizar pedido por QR	La mesa tiene un código QR asignado.	Escanear QR -> identificar mesa -> mostrar menú digital -> seleccionar productos y	Pedido asociado a la mesa con estado “pendiente”.

		cantidades -> confirmar -> validar -> guardar pedido -> enviar a cocina.	

9.3 Caso de uso: Gestionar preparación del pedido

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Gestionar preparación del pedido	Existen pedidos pendientes registrados.	Listar pendientes -> seleccionar pedido -> validar secuencia -> cambiar estado -> actualizar pedido.	Cambio de estado guardado y visible para el personal y el cliente.

9.4 Caso de uso: Cobrar pedido en la caja

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Cobrar pedido en la caja	El pedido está en estado listo.	Buscar pedido por mesa -> validar que esté listo -> mostrar detalle y total -> registrar método de pago -> actualizar pedido.	Pedido “pagado” y habilitado como “listo para entregar”.

10. Procedimiento recomendado para el estudiante

- Revisar la interfaz CRUD e identificar campos de datos: ID, Nombre y Edad.
- Redactar requisitos funcionales de nivel 0 para Agregar, Actualizar, Eliminar y Mostrar Todo.
- Modelar los casos de uso en PlantUML y justificar únicamente las relaciones include necesarias.
- Derivar clases de análisis: una clase frontera para la interfaz, una clase control para coordinar la lógica, una entidad Estudiante y un repositorio o colección.
- Construir un diagrama de secuencia por cada caso de uso principal.
- Verificar que cada mensaje del diagrama de secuencia corresponda a una operación declarada en el diagrama de clases.
- Actualizar la matriz de trazabilidad para demostrar coherencia entre RF, CU, clases y secuencias.

11. Lista de verificación de coherencia

- Cada requisito funcional tiene al menos un caso de uso asociado.
- Cada caso de uso principal posee una secuencia de análisis propia.
- Las clases usadas en las secuencias existen en el diagrama de clases de análisis.
- Los mensajes de secuencia coinciden con operaciones de las clases de análisis.
- La entidad Estudiante conserva los atributos ID, Nombre y Edad.
- La validación de datos se aplica antes de agregar y actualizar.
- El diagrama no mezcla detalles de implementación de interfaz gráfica con responsabilidades de análisis.

12. Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad entre modelos	Relaciona correctamente RF, casos de uso, clases y secuencias.	5 pts	Matriz de relación completa.
Diagrama de casos de uso	Representa actor, sistema, casos de uso y relaciones include justificadas.	4 pts	Código y diagrama PlantUML.
Diagrama de clases de análisis	Identifica frontera, control, entidad y repositorio con responsabilidades claras.	4 pts	Código PlantUML de clases.
Diagramas de secuencia	Modela flujos coherentes para Agregar, Actualizar,	5 pts	Cuatro secuencias PlantUML.

	Eliminar y Mostrar Todo.		
Presentación técnica	Documento ordenado, lenguaje claro y sin errores de sintaxis PlantUML.	2 pts	Entrega final documentada.

13. Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Casos de uso	Requisitos funcionales, diagrama CU y especificación breve.	Código PlantUML + captura.
Clases de análisis	Diagrama con boundary, control, entity y repositorio.	Código PlantUML + captura.
Secuencias de análisis	Una secuencia por cada operación CRUD principal.	Códigos PlantUML + capturas.
Trazabilidad	Matriz que conecte RF, CU, clases y secuencias.	Tabla de relación.
Conclusión técnica	Reflexión breve sobre la coherencia entre modelos.	Texto de cierre.

14. Conclusión técnica

La relación entre casos de uso, clases de análisis y diagramas de secuencia permite comprobar que el modelo de Bocatto no se construye como piezas aisladas. Los casos de uso explican qué necesita cada actor; las clases de análisis explican qué objetos colaboran; y las secuencias muestran cómo esos objetos interactúan para cumplir cada caso de uso. A continuación se resume lo que ocurre en cada uno.

14.1 Caso de uso: Tomar pedido en mesa

Elemento	Descripción
Actor	Mesero
Qué ocurre	El mesero selecciona la mesa y registra productos, cantidades y observaciones; el sistema valida que exista al menos un producto, crea el pedido en estado “pendiente” y lo envía a cocina.
Clases que colaboran	PantallaPedido, ControlPedido, Mesa, Producto, Pedido, RepositorioPedido y PantallaCocina.
Aporte del repositorio	RepositorioPedido guarda el pedido registrado para que cocina y caja puedan consultarlo.

14.2 Caso de uso: Realizar pedido por QR

Elemento	Descripción
Actor	Cliente
Qué ocurre	El cliente escanea el QR de su mesa, el sistema identifica la mesa y muestra el menú digital; al confirmar, el pedido se valida, queda en “pendiente” y pasa a cocina sin intervención del mesero.
Clases que colaboran	PantallaMenuDigital, ControlPedidoQR, Mesa, Producto, Pedido, RepositorioPedido y PantallaCocina.
Aporte del repositorio	RepositorioPedido guarda el pedido realizado por el cliente.

14.3 Caso de uso: Gestionar preparación del pedido

Elemento	Descripción
Actor	Cocinero
Qué ocurre	El cocinero revisa los pedidos pendientes y cambia su estado en orden (en preparación, listo, listo para entregar); cada cambio se guarda y lo ven el personal y el cliente.
Clases que colaboran	PantallaCocina, ControlPreparacion, Pedido y RepositorioPedido.
Aporte del repositorio	RepositorioPedido entrega la lista de pendientes y guarda cada cambio de estado.

14.4 Caso de uso: Cobrar pedido en la caja

Elemento	Descripción
Actor	Cajero
Qué ocurre	El cajero busca el pedido por número de mesa, verifica que esté listo, revisa el detalle y el total y registra el método de pago; el pedido queda “pagado” y habilitado para la entrega.
Clases que colaboran	PantallaCaja, ControlCobro, Mesa, Pedido y RepositorioPedido.
Aporte del repositorio	RepositorioPedido permite buscar el pedido por mesa y guarda el pago registrado.

En los cuatro casos el repositorio conserva la información del pedido, lo que mantiene la trazabilidad desde el requisito funcional hasta la interacción entre objetos y prepara el diseño posterior del sistema.

15. Fuente interna de referencia

- 01_Guia_PlantUML_CU_Analisis_CRUD_Estudiante.
- Guía Version2 del CRUD Estudiante con campos ID, Nombre y Edad, adaptada a la interfaz de referencia proporcionada por la docente.